

■ IL PROCESSO PERSONALE OTTIMALE PER UNA FEATURE

(Bug-Free Development Workflow)

Basato su ricerca scientifica di oltre 4 anni su 23.000+ developer

FASE 1: COMPrensione PROFONDA (30-40% del tempo totale)

Questo è il passo che i junior saltano e i senior lo fanno ossessivamente.

1.1 Leggi il problema 3 volte

- Leggi completamente la user story
- Apri e leggi documenti referenziati (RFCs, docs, code snippets)
- Se è un'estensione di feature esistente, testala prima
- Non cercare ancora di capire il codice. Solo il **cosa** devi fare.

1.2 Fai domande esplicite (PRIMA di codare)

- Chiarisci: scope, use cases, personas, casi di errore, edge cases
- Cosa succede se input è null?
- Cosa succede se server è lento?
- Cosa succede se l'utente double-click?
- Comunica con chi capisce il problema in profondità (clienti, stakeholder, utenti finali)

1.3 Gioca con il sistema PRIMA di codare

Se è una feature nuova in un'app esistente: interagiscici, testa diverse flow, vedi i limiti. Scopri come l'utente userà la feature nella realtà, non in teoria.

1.4 Creare il MENTAL MODEL (Cruciale)

Non cercare di capire la codebase intera. Build a rough mental model prima e refina nel tempo. Identifica:

- Major components
- Dove i dati entrano
- Dove sono stored
- Quali file coordinano behavior

La 'scala' mentale per capire il codice:

1. Cos'è il README e la structure del progetto? 2. Dove avviene il flusso principale/critico? 3. Quali componenti toccherò con questa feature? 4. Quali funzioni/moduli devo modificare/creare? 5. Quali dipendenze hanno?

FASE 2: DESIGN PRIMA DI CODARE (20-25%)

2.1 Scrivi pseudocode/comments in plain English

Prima di scrivere codice vero, mappa la logica in plain English o comments. Questo clarifica i tuoi pensieri e spot gaps early:

```
// Check if user is logged in  
// If not, redirect to login page  
// If logged in, fetch user profile  
// Handle loading and error states  
// If error, show fallback UI
```

2.2 Identify ALL edge cases PRIMA

La parte più difficile del job del developer è identificare i edge cases - situazioni che nemmeno l'utente pensa accadranno.

Domande da farti:

• Input vuoto/null/undefined? • Input fuori range? • Dipendenze esterne falliscono (API, DB)? • User interazioni concorrenti (double-click, rapid fire)? • Stato inconsistente? • Permessi non validi? • Rate limits? • Condizioni di gara (race conditions)?

Scrivi queste down - non nel tuo mental. Saranno la base per i test.

2.3 Specifica input/output chiaramente

• Quali input accetto? • Quali output ritorno? • Quali errori posso avere? • Quali assunzioni sto facendo?

FASE 3: DEFENSIVE PROGRAMMING MINDSET

(Before Writing Code)

Non sarai mai meglio al debugging del tuo codice quando lo scrivi per la prima volta. Devi scrivere codice per cose che NON ti aspetti accadranno o NON dovrebbero accadere.

3.1 Assumption checking

Non assumere che inputs sono sempre validi o within range. Il tuo codice deve handle anomalie gracefully, predictably, safely.

```
// ■ ASSUME (Junior)
function processOrder(order) {
  const total = order.items.reduce((sum, item) =>
    sum + item.price, 0);
}

// ■ DEFENSIVE (Senior)
function processOrder(order) {
  if (!order) throw new Error("Order is required");
  if (!Array.isArray(order.items) ||
    order.items.length === 0) {
    throw new Error("Order must have items");
  }
  const total = order.items.reduce((sum, item) => {
    if (typeof item.price !== 'number' ||
      item.price < 0) {
      throw new Error(`Invalid price: ${item.price}`);
    }
    return sum + item.price;
  }, 0);
  return total;
}
```

3.2 Validazione esplicita

- Check array/string bounds prima di accedere
- Sempre validate indices e pointers
- Handle exceptions esplicitamente e consistentemente
- Fail safe quando errors occorrono

3.3 Assertions (durante dev)

Assertions sono runtime checkpoints che verificano che certe condizioni, che credi debbano essere true, in realtà lo sono. Se falliscono, feedback immediato:

```
def normalize_data(min_val, max_val, values):
  assert min_val < max_val,
    f"min ({min_val}) must be < max ({max_val})"
  assert all(isinstance(v, (int, float)) for v in values),
    "All values must be numbers"
  # implementation...
```

FASE 4: TEST-FIRST / SPEC-FIRST (15-20%)

Questo è il game-changer per bug reduction.

4.1 Scrivi test PRIMA del codice

Automated unit e acceptance tests dovrebbero girare contro ogni commit. Nessuno dice 'done' finché tutti i relevant automated tests non passano:

```
describe('calculateTotal', () => {
  it('should sum valid items', () => {
    const order = { items: [{price: 10}, {price: 20}] };
    expect(calculateTotal(order)).toBe(30);
  });

  it('should handle null order gracefully', () => {
    expect(() => calculateTotal(null))
      .toThrow('Order required');
  });

  it('should reject negative prices', () => {
    const order = { items: [{price: -10}] };
    expect(() => calculateTotal(order))
      .toThrow('Invalid price');
  });

  it('should handle empty items array', () => {
    const order = { items: [] };
    expect(() => calculateTotal(order))
      .toThrow('Must have items');
  });
});
```

4.2 Test coverage deve includere:

- Happy path (funziona tutto perfetto)
- Error cases (inputs invalidi)
- Edge cases (null, empty, extreme values)
- Integration points (dipendenze esterne)

Statistica: Teams che conducono testing hanno il 30% fewer post-launch bugs.

FASE 5: IMPLEMENTAZIONE INCREMENTALE (20-25%)

Non scrivere tutto in una volta. Small steps atomici.

5.1 Implementa per piccoli pezzi atomici

- Scrivi 10-20 linee di codice
- Runna i test → deve passare
- Ripeti il ciclo

Principio della small batch: Short-lived branches (<1 day), frequent commits. Minimizza l'impatto che un errore potrebbe causare.

5.2 Refactor mentre implementi

- Non aspettare la fine
- Se vedi codice brutto, refactor subito
- Mantieni mental clarity

5.3 Usa assertions durante dev

- Lascia i tuoi defensive checks nel codice
- Assertions falliranno subito se assunzioni sono violate
- Feedback immediato = meno debugging dopo

FASE 6: TESTING PERSONALE PRIMA DEL COMMIT

(10-15%)

6.1 Test manualmente gli edge cases che hai identificato

- Copia/incolla i tuoi edge cases dalla Fase 2
- Testa UNO PER UNO nella realtà
- Non skip nemmeno uno
- Documenta cosa hai testato

6.2 Pensa come un attaccante

Non anticipare come il tuo codice potrebbe essere misused è il motore di vulnerabilità. Ask: 'How could this break?'

6.3 Code review a te stesso

Leggi il codice come se l'avesse scritto qualcun altro:

- Sono i nomi chiari e descrittivi?
- La logica è ovvia al primo sguardo?
- Mancano sanity checks/validazioni?
- Ci sono assunzioni nascoste?
- Gli errori sono gestiti completamente?
- C'è code duplication che dovrebbe essere refactored?

■ FLUSSO VISUALE (Per una feature medium)

FASE	PERCENTUALE	DURATA (ES.)	ATTIVITÀ PRINCIPALE
UNDERSTAND	40%	5h	Reading spec, Ask questions, Test codebase
DESIGN	25%	3h	Identify edge cases, Plan algorithm, Defensive checks
TESTS	15%	2h	Write test cases, Test manually
CODE	20%	4h	Implement code, Refactor, Test
VERIFY	~5%	1h	Final checks, Code review self

■ GLI ERRORI CHE FANNO I JUNIOR

(E che tu devi evitare)

■ Saltare il 'Plan Your Approach'

Jumping straight to coding based on assumptions (spesso sbagliati). Questo è il #1 bug generator.

■ Costruire quello che è descritto, non quello che è needed

Clienti non sanno esattamente cosa vogliono al Day 1. Una volta che iniziano a usare il codice, si rendono conto che manca qualcosa o che è sbagliato.

■ Velocità di scrittura vs tempo di pensiero

I migliori developer spendono PIÙ tempo a pensare che a tipare. Se passi il 80% del tempo a scrivere e il 20% a pensare, invertire questo rapporto ridurrà i bug drasticamente.

■ Non comunicare con chi capisce il dominio

Le assunzioni diventano bug. Parla con business stakeholder, product manager, utenti. Ogni ora spesa qui risparmia giorni di rework.

■ Testare solo il happy path

Se testi solo il caso 'perfetto', scoprirai i bug quando sono in produzione e costosi. Test i 'forbidden cases' per primo.

■ CHECKLIST PERSONALE PER OGNI FEATURE

PRIMA DI SCRIVERE UNA LINEA DI CODICE:

■ Ho letto il requirement 3+ volte? ■ Ho fatto tutte le domande chiarificative necessarie? ■ Ho testato il sistema esistente (se applicabile)? ■ Ho mappato il codebase (costruito mental model)? ■ Ho scritto pseudocode della soluzione? ■ Ho identificato TUTTI i possibili edge cases? ■ Ho documentato input/output/errors chiaramente? ■ Ho pianificato la defensive programming strategy?

DURANTE IMPLEMENTAZIONE:

■ Scrivo test PRIMA del codice (per ogni test case)? ■ Implemento in piccoli step atomici (10-20 linee)? ■ Runno test dopo ogni step? ■ Lascio assertions nel codice per debug? ■ Refactor quando sento odore di codice cattivo? ■ Mantengo codice semplice e readable?

PRIMA DI ANDARE IN PRODUZIONE:

■ Tutti i test automatici passano? ■ Ho testato manualmente ogni edge case? ■ Ho fatto code review a me stesso? ■ Ho verificato error handling completo? ■ Ho documentato decisioni non-ovvie? ■ Ho fatto testing in ambiente simile a prod? ■ Ho pianificato rollback se qualcosa va male?

■ IL SECRET SAUCE: Mental Simulation

I developer 10x corrono mental simulations di alternative scenarios prima di implementare. Questo significa predicting issues prima che occorran e understanding systems più profondamente.

Prima di codare, simula mentalmente:

• Cosa succede se input X viene passato? • Cosa succede se dipendenza Y fallisce? • Cosa accade se due utenti concurrent accedono? • Quale è il best case scenario? • Quale è il worst case scenario? • Come il sistema degrada quando c'è errore?

Questa simulation mentale è quello che separa i developer che scrivono codice buggy da quelli che non lo fanno.

Tempo Totale Approssimativo per una Feature Medium:

Understand: 5 ore (40%) Design: 3 ore (25%) Tests: 2 ore (15%) Code: 4 ore (20%) Verify: 1 ora (~5%)  TOTALE: ~15 ore

Se il tuo problema è complessità, non velocità, aumenta le ore di UNDERSTAND e DESIGN.
È il migliore ROI per ridurre bug.

Documento basato su ricerca DORA, Accelerate (Forsgren, Humble, Kim), studi su defensive programming, e 15+ anni di best practices da developer senior.